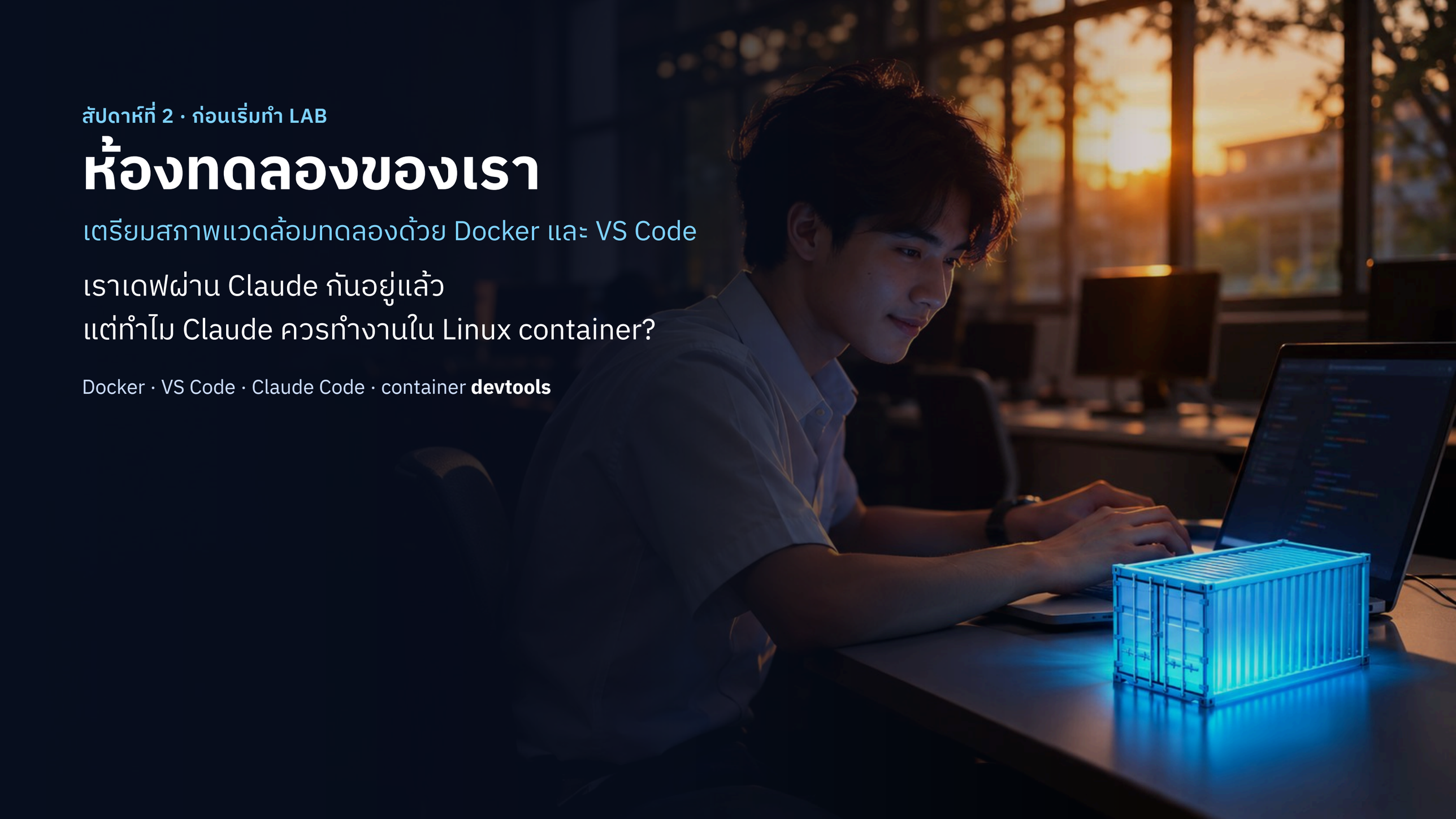
สัปดาห์ที่ 2 · ก่อนเริ่มทำ LAB

ห้องทดลองของเรา

เตรียมสภาพแวดล้อมทดลองด้วย Docker และ VS Code

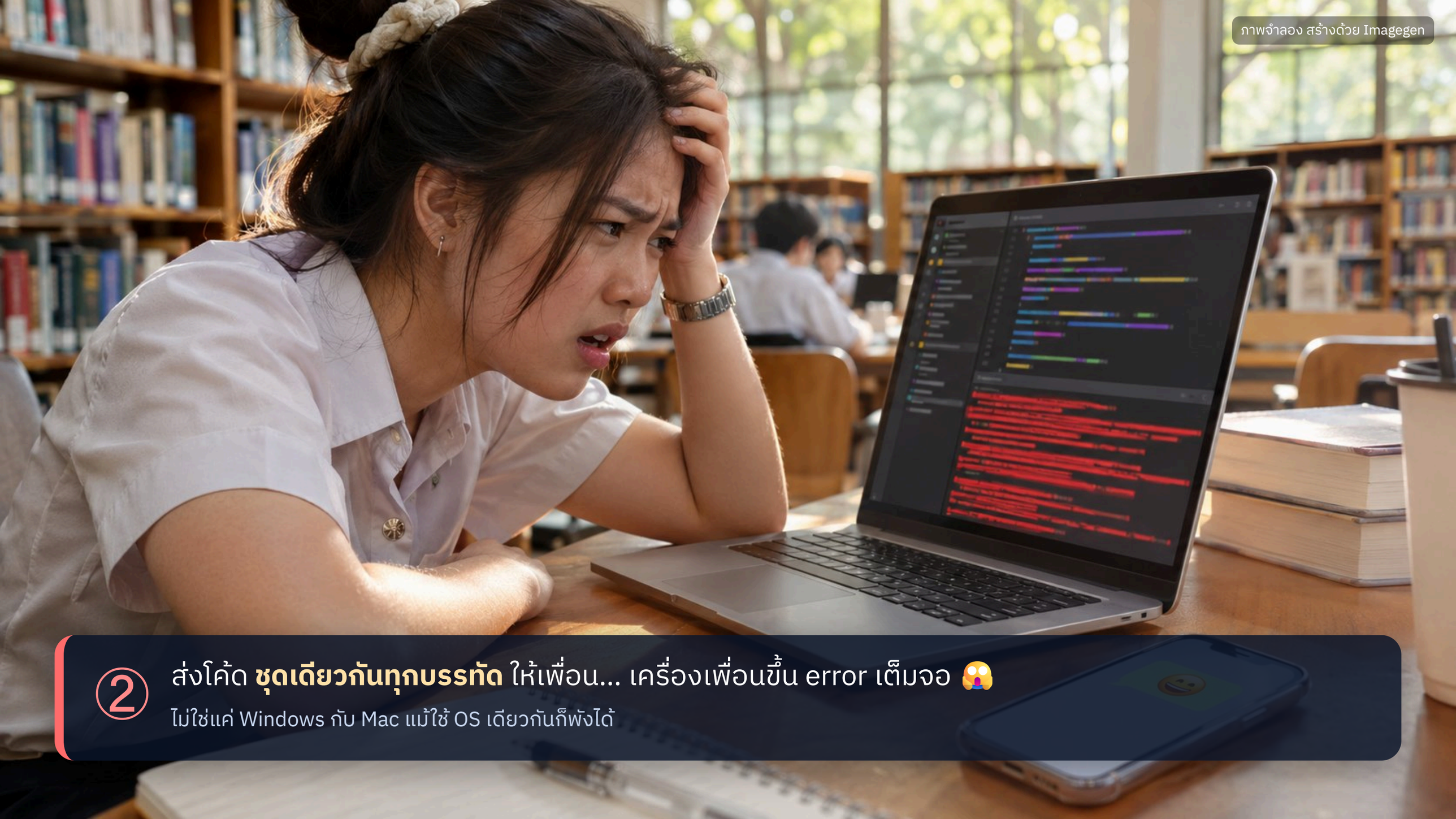
เราเดฟผ่าน Claude กันอยู่แล้ว
แต่ทำไม Claude ควรทำงานใน Linux container?

Docker · VS Code · Claude Code · container **devtools**





① เราให้ Claude Code สร้างแอป Python จำแนกรูปภาพ ด้วยโมเดลจาก Hugging Face **บนเครื่องเรา... รันได้!** 🎉



② ส่งโค้ด **ชุดเดียวกันทุกบรรทัด** ให้เพื่อน... เครื่องเพื่อนขึ้น error เต็มจอ 😱

ไม่ใช่แค่ Windows กับ Mac แม้ใช้ OS เดียวกันก็พังได้

ทำไม? โค้ดเหมือนกันเป๊ะ

โค้ดเหมือนกัน แต่ "พื้น" ที่โค้ดยืนอยู่ไม่เหมือนกัน

เครื่องเรา ✅ รันได้

- app.py · แอปจำแนกรูป (โค้ดเดียวกัน)
- transformers **4.46** · torch **2.5**
- Python **3.12**
- Windows 11

≠

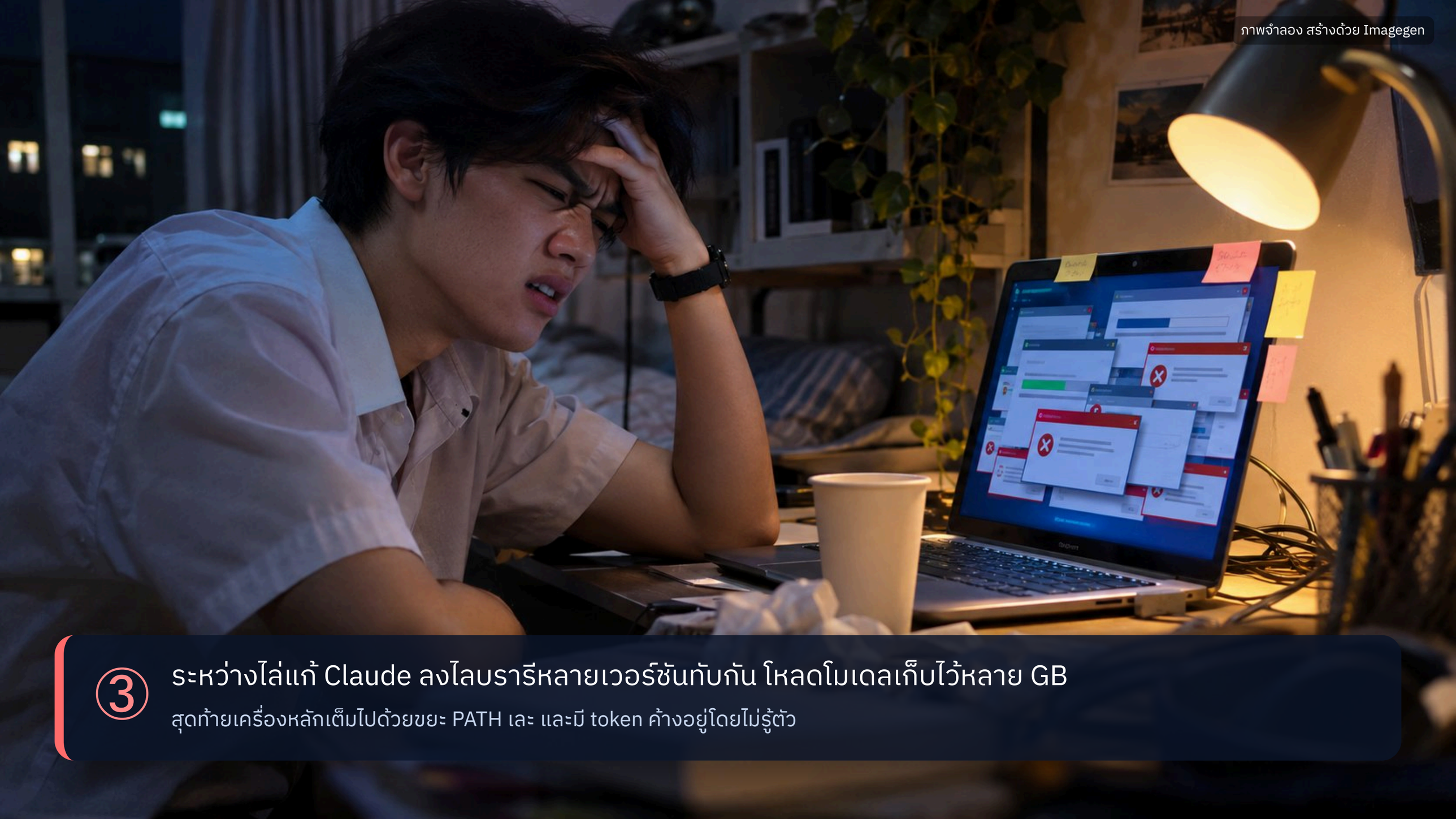
เครื่องเพื่อน ❌ error

- app.py · แอปจำแนกรูป (โค้ดเดียวกัน)
- transformers **4.30** · torch **2.1**
- Python **3.9**
- macOS (หรือ Windows อีกเครื่อง)

Claude ใช้ Python และไลบรารีที่ **เครื่องนั้นมีอยู่** หรือติดตั้งเวอร์ชันที่หาได้ตอนนั้น แต่ละเครื่องจึงได้ชุดเวอร์ชันไม่เหมือนกัน

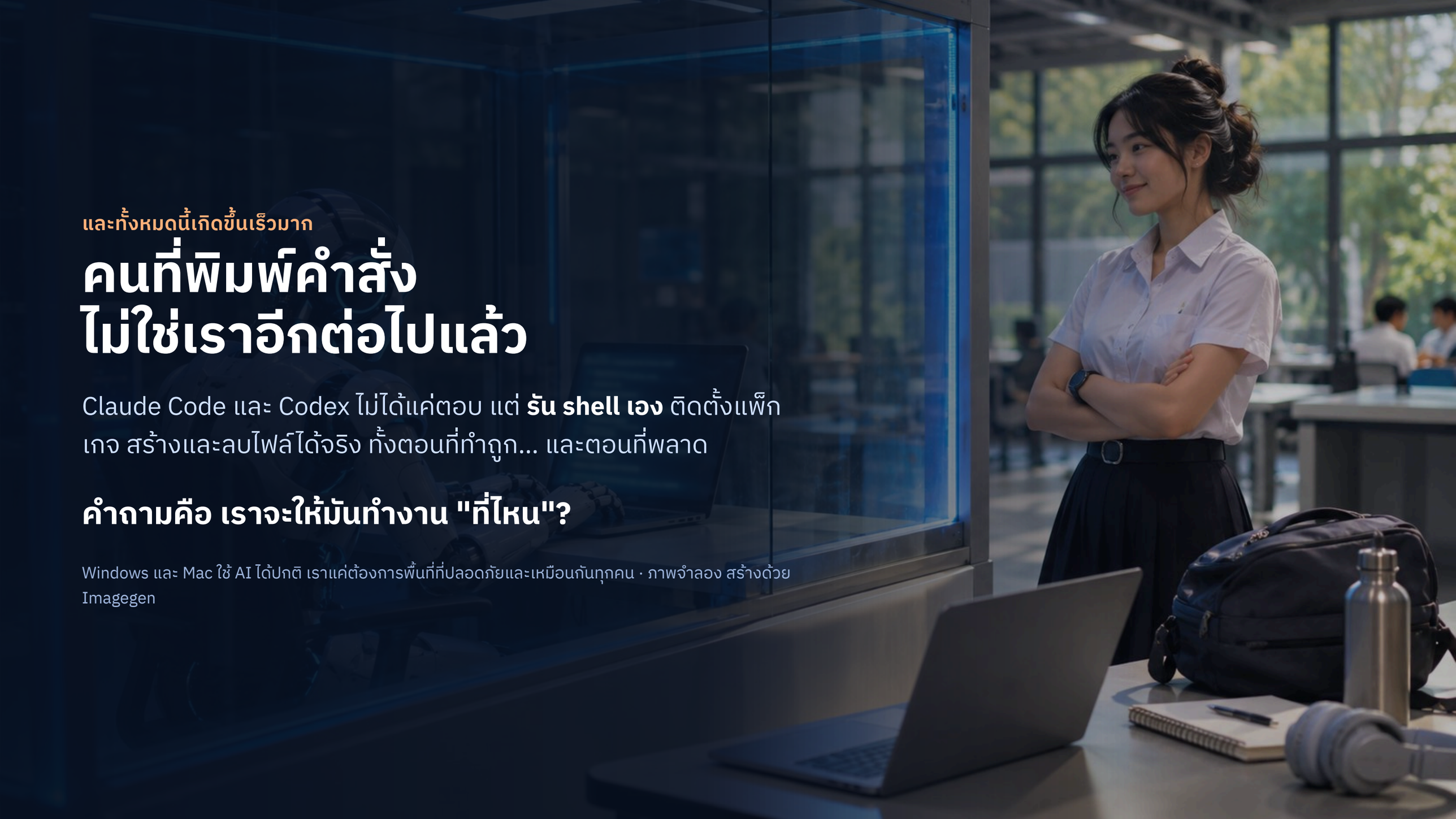
ชั้นสีแดงคือส่วนที่ต่างกัน แค่ชั้นเดียวไม่ตรง แอปก็พังได้ ต่อให้ Claude เก่งแค่ไหนก็ต้องมานั่งไล่แก้ทีละเครื่อง

เลขเวอร์ชันเป็นตัวอย่างประกอบ



③ ระหว่างไล่แก้ Claude ลงไลบรารีหลายเวอร์ชันทับกัน โหลดโมเดลเก็บไว้หลาย GB

สุดท้ายเครื่องหลักเต็มไปด้วยขยะ PATH เละ และมี token ค้างอยู่โดยไม่รู้ตัว



และทั้งหมดนี้เกิดขึ้นเร็วมาก

คนที่พิมพ์คำสั่ง ไม่ใช่เราอีกต่อไปแล้ว

Claude Code และ Codex ไม่ได้แค่ตอบ แต่ **รัน shell เอง** ติดตั้งแพ็กเกจ สร้างและลบไฟล์ได้จริง ทั้งตอนที่ทำถูก... และตอนที่พลาด

คำถามคือ เราจะให้มันทำงาน "ที่ไหน"?

Windows และ Mac ใช้ AI ได้ปกติ เราแค่ต้องการพื้นที่ที่ปลอดภัยและเหมือนกันทุกคน · ภาพจำลอง สร้างด้วย Imagegen



4 ถ้าทุกคนให้ Claude ทำงานใน **Linux container เดียวกัน**
โค้ดชุดเดิม... รันได้ทั้งสองเครื่องทันที ✅ และพังเมื่อไรก็ลบแล้วสร้างใหม่ได้

หัวใจของเรื่องนี้

ผลลัพธ์ไม่ได้ขึ้นกับโค้ดอย่างเดียว

❌ ต่างคนต่างรันบนเครื่องตัวเอง

📄 โค้ดเดียวกัน + 🎲 environment ไม่เหมือนกัน = 💥 ผลไม่ตรงกัน

✅ ทุกคนรันใน container เดียวกัน

📄 โค้ดเดียวกัน + 🐳 environment เดียวกัน = ✅ ผลเหมือนกัน

🤝 ส่งโค้ดและพรอมต์ให้เพื่อน รันได้เลย

🧪 Claude และขยะทั้งหมดอยู่ในกล่อง

🔁 พังแล้วลบ สร้างใหม่ได้ในไม่กี่นาที

"environment เดียวกัน" นี้คือ... →

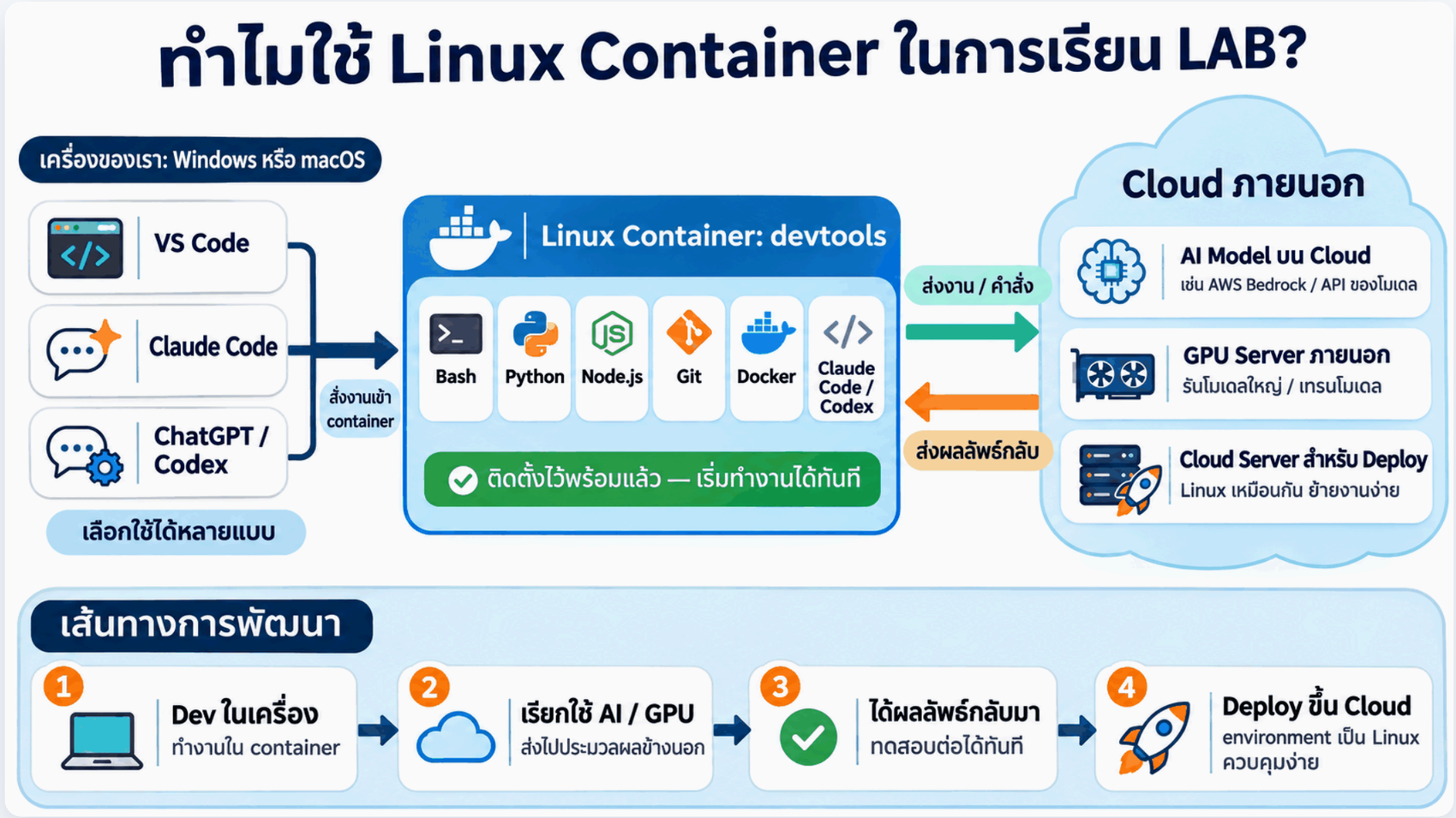


คำตอบคือ... Linux container

"ห้องทดลอง" Linux ที่ติดตั้งเครื่องมือไว้พร้อม รันบนเครื่องของเราผ่าน Docker Desktop (Windows ใช้ WSL 2 ส่วน Mac ก็มี Docker Desktop) ทุกคนได้ Linux ชุดเดียวกัน แล้วใช้ VS Code ที่คุ้นเคยเข้าไปทำงานข้างใน

คำตอบ

ใช้เครื่องมือไหนก็ได้ → container เดียวกัน → ต่อยอดไปข้างนอก



ภาพแนวคิด สร้างด้วย Imagegen: Windows หรือ macOS จะสั่งงานผ่าน VS Code, Claude Code หรือ ChatGPT / Codex ก็ได้ ทุกทางเข้า container เดียวกันที่มี Bash, Python, Node.js, Git, Docker, Claude Code / Codex พร้อมใช้ และส่งงานไป AI model, GPU server หรือ deploy ขึ้น cloud ได้

คำตอบ

ปัญหาเดิม หายไปไหน?

	❌ ไม่มี container	✅ มี container devtools
เครื่องมือ	ทุกคนติดตั้งเอง เวอร์ชันไม่ตรงกัน	เริ่มจาก image ของวิชาเดียวกันทุกคน
คำสั่ง LAB	ต้องแปลคำสั่ง Linux เป็นของ Windows	เป็น Linux อยู่แล้ว ก๊อปวางได้เลย
ทดลองพลาด	เครื่องหลักพัง ลงใหม่หลายชั่วโมง	ลบ container แล้วสร้างใหม่จาก image
ให้ AI รันคำสั่ง	รันตรงบนเครื่องหลัก	รันในกล่องแยก พลาดก็รีเซ็ตได้
ส่งงานข้าม OS	Windows ↔ Mac ต้องแปลงบรรทัด พาธ สคริปต์	ทุกคนอยู่ใน Linux เดียวกัน แชร์แล้วรันได้เลย
ขยะจาก AI	กระจายในเครื่องหลัก ลบไม่หมด	อยู่ใน container ลบทีเดียวหมด
ไปต่อ cloud	environment ตอน dev ต่างจากตอน deploy	Linux เหมือน server ส่วนใหญ่ ย้ายงานง่าย

แยกพื้นที่ได้ดีขึ้น แต่ไม่ได้รับประกัน 100% โดยเฉพาะชุดนี้ที่ใช้ `--privileged` เพื่อให้ใช้ Docker ข้างใน container ได้ และถ้าเชื่อมโฟลเดอร์จากเครื่องหลักเข้าไป ไฟล์นั้นก็ถูกแก้ได้

ทำไมรีเซ็ตได้

Image = แม่พิมพ์ · Container = ชิ้นงานที่ปั๊มออกมา



Image คือแม่พิมพ์ที่ไม่เปลี่ยน เช่น `tuchsanai/devtools:2569_1`

Container คือชิ้นงานที่ปั๊มจาก image แต่ละชิ้นตกแต่งเพิ่มเองได้ (ติดตั้งของ แก้ไฟล์)

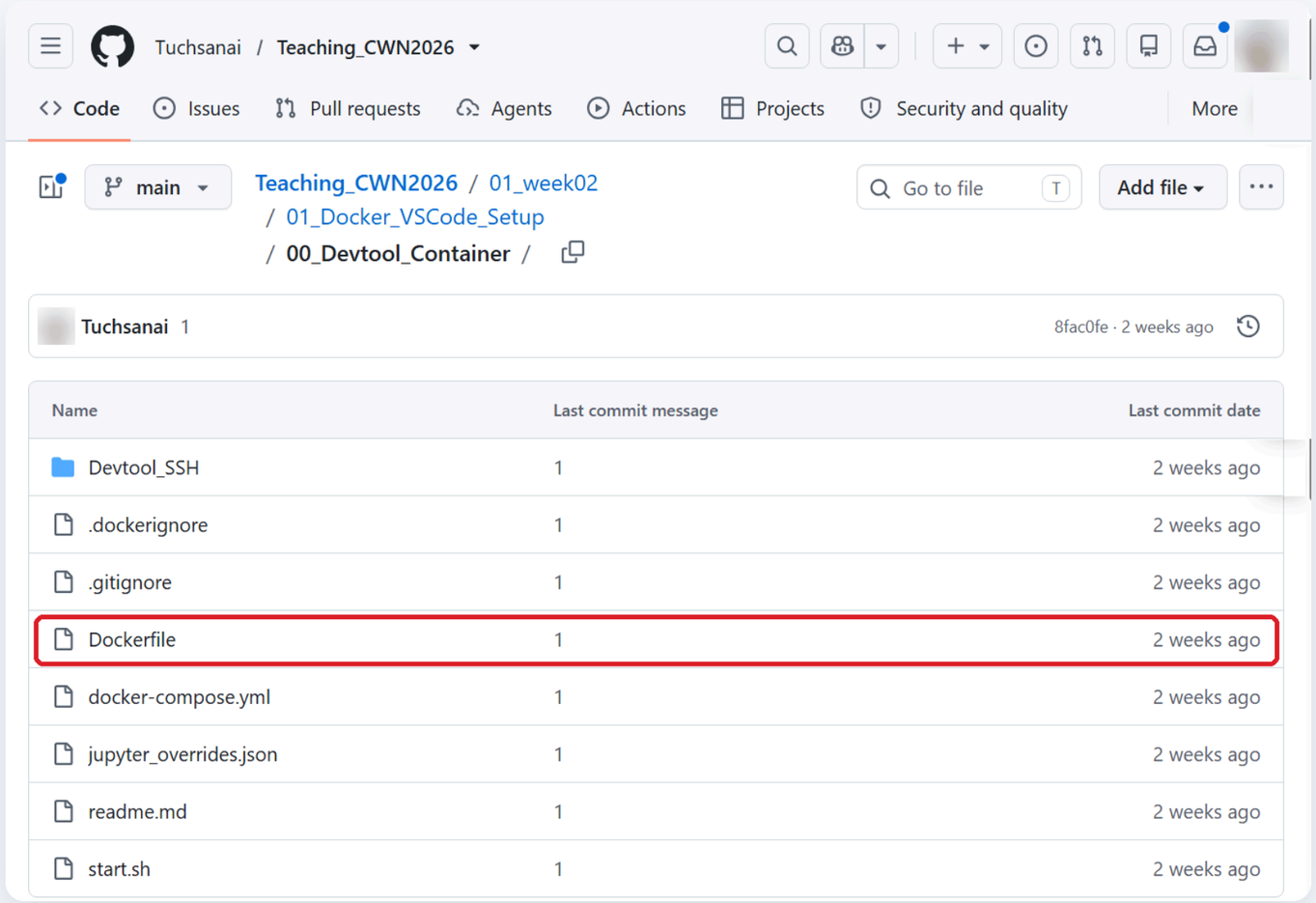
ชิ้นไหนเสีย **ทิ้งแล้วปั๊มใหม่** ได้ทันที แม่พิมพ์ยังอยู่ครบ

> ปั๊มใหม่ = กลับไปเหมือนแม่พิมพ์ ของที่แต่งไว้ในชิ้นเดิมไม่ตามมา ไฟล์ใน container หายเมื่อลบ งานสำคัญเก็บด้วย Git / volume / backup

ภาพเปรียบเทียบ (ภาพจำลอง สร้างด้วย Imagegen)

แม่พิมพ์ของวิชา

แล้วแม่พิมพ์ devtools มาจากไหน?



ภาพจริงจากหน้า GitHub ของวิชา กรอบแดงคือ Dockerfile (เบลออวาตาร์บัญชีไว้)

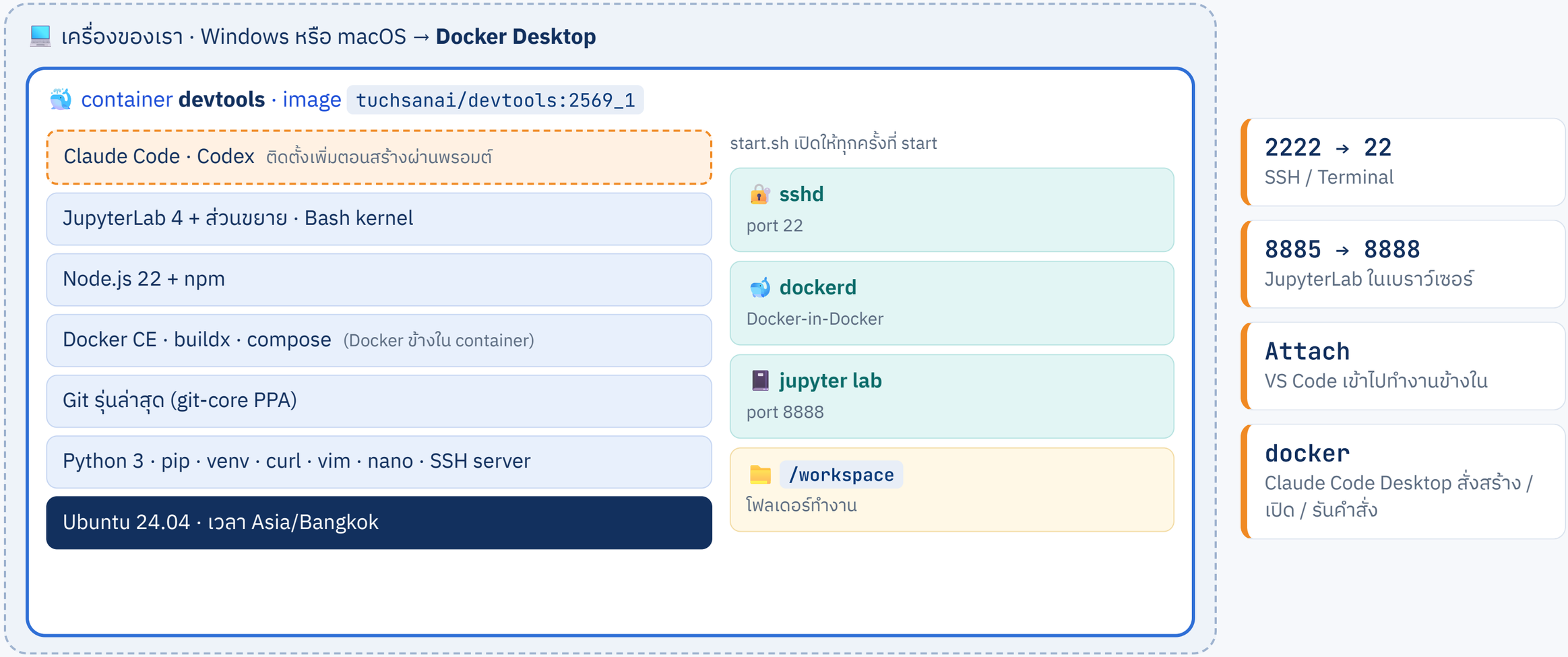
มาจากโฟลเดอร์ `00_Devtool_Container` ในรีโปของวิชา ทุกคนเปิดดูได้

- **Dockerfile** สูตรทีละขั้น: เริ่มจาก Ubuntu 24.04 แล้วติดตั้ง Python, Node.js, Git, Docker-in-Docker, JupyterLab, SSH
- **start.sh** เปิดบริการทั้งหมดทุกครั้งที่ container เริ่มทำงาน
- **docker-compose.yml** ทางเลือก build + run คำสั่งเดียว

> สูตรเปิดเผยหมด จึงรู้ว่าในห้องทดลองมีอะไรบ้าง และทำซ้ำได้เหมือนกันทุกเครื่อง

ข้างในห้องทดลอง

เปิดกล่อง devtools ดูกัน: Linux ธรรมดาที่ลงโปรแกรมไว้ครบ



วาดจาก Dockerfile และ start.sh จริงในโฟลเดอร์ 00_Devtool_Container · image นี้เป็น `linux/amd64` บน Mac ชิป Apple Silicon Docker Desktop รันผ่านการจำลองได้ แต่อาจช้ากว่า

ข้างในห้องทดลอง

สูตรจริงใน Dockerfile (ตัดมาเฉพาะส่วนสำคัญ)

```
FROM ubuntu:24.04                                  # ①
ENV TZ=Asia/Bangkok LANG=C.UTF-8

RUN apt-get install -y curl wget vim nano git \      # ②
      openssh-server python3 python3-pip python3-venv …
RUN add-apt-repository -y ppa:git-core/ppa …         # ③
RUN … apt-get install -y docker-ce docker-ce-cli \   # ④
      containerd.io docker-buildx-plugin docker-compose-plugin
RUN curl -fsSL https://deb.nodesource.com/setup_22.x | bash - \
 && apt-get install -y nodejs                          # ⑤
RUN pip install "jupyterlab>=4.3,<5" jupytext jupyterlab-git … # ⑥
RUN echo 'root:passwd' | chpasswd …                  # ⑦
COPY start.sh /usr/local/bin/start.sh
EXPOSE 22 8888
WORKDIR /workspace
CMD ["/usr/local/bin/start.sh"]                      # ⑧
```

① เริ่มจาก Ubuntu 24.04 ตั้งเวลาไทย

② เครื่องมือพื้นฐาน + Python + SSH server

③ Git รุ่นล่าสุด (ใช้ git worktree ได้ครบ)

④ Docker ข้างใน container (Docker-in-Docker)

⑤ Node.js 22 สำหรับเว็บ Next.js / React / Vite

⑥ JupyterLab และส่วนขยาย

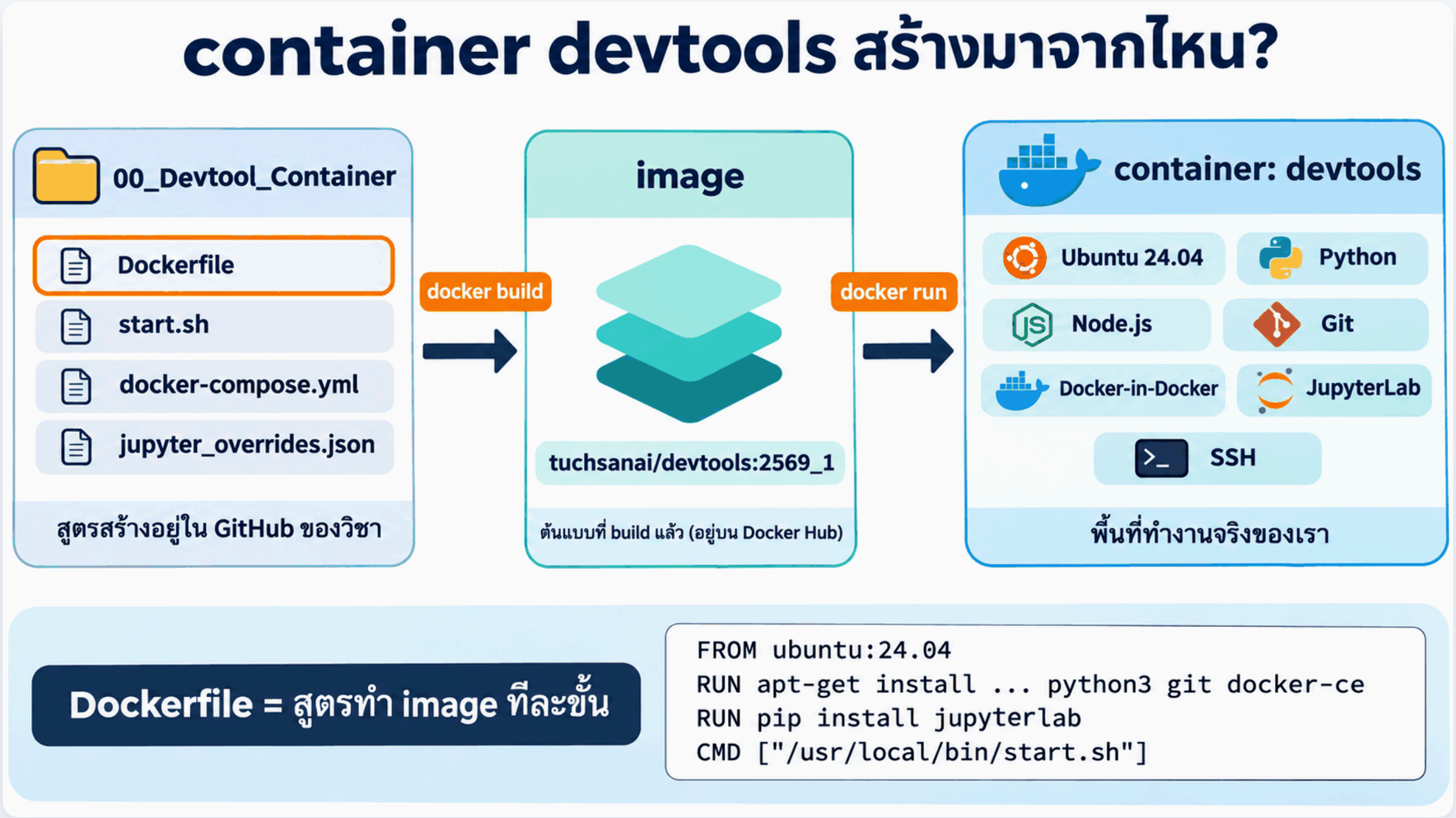
⑦ บัญชี root สำหรับ SSH (รหัสเดียวกันทุกเครื่อง ใช้เรียนบนเครื่องตัวเองเท่านั้น)

⑧ ทุกครั้งที่ start รัน start.sh เปิด sshd, dockerd, JupyterLab

ฉบับเต็มอยู่ที่ 00_Devtool_Container/Dockerfile · เครื่องหมาย … คือส่วนที่ตัดออกเพื่อให้อ่านง่าย

แม่พิมพ์ของวิชา

สูตร → แม่พิมพ์ → ห้องทดลอง



อาจารย์ **build** สูตรเป็น image `tuchsanai/devtools:2569_1` ไว้บน Docker Hub แล้ว ผู้เรียนแค่ **run** ก็ได้ container ทันที อยากลอง build เองดู readme ของโฟลเดอร์ (ภาพแนวคิด สร้างด้วย Imagegen)

มองไกลกว่าห้องเรียน

ห้องทดลองนี้ คือฐานทำงานกลางของทั้งเทอม

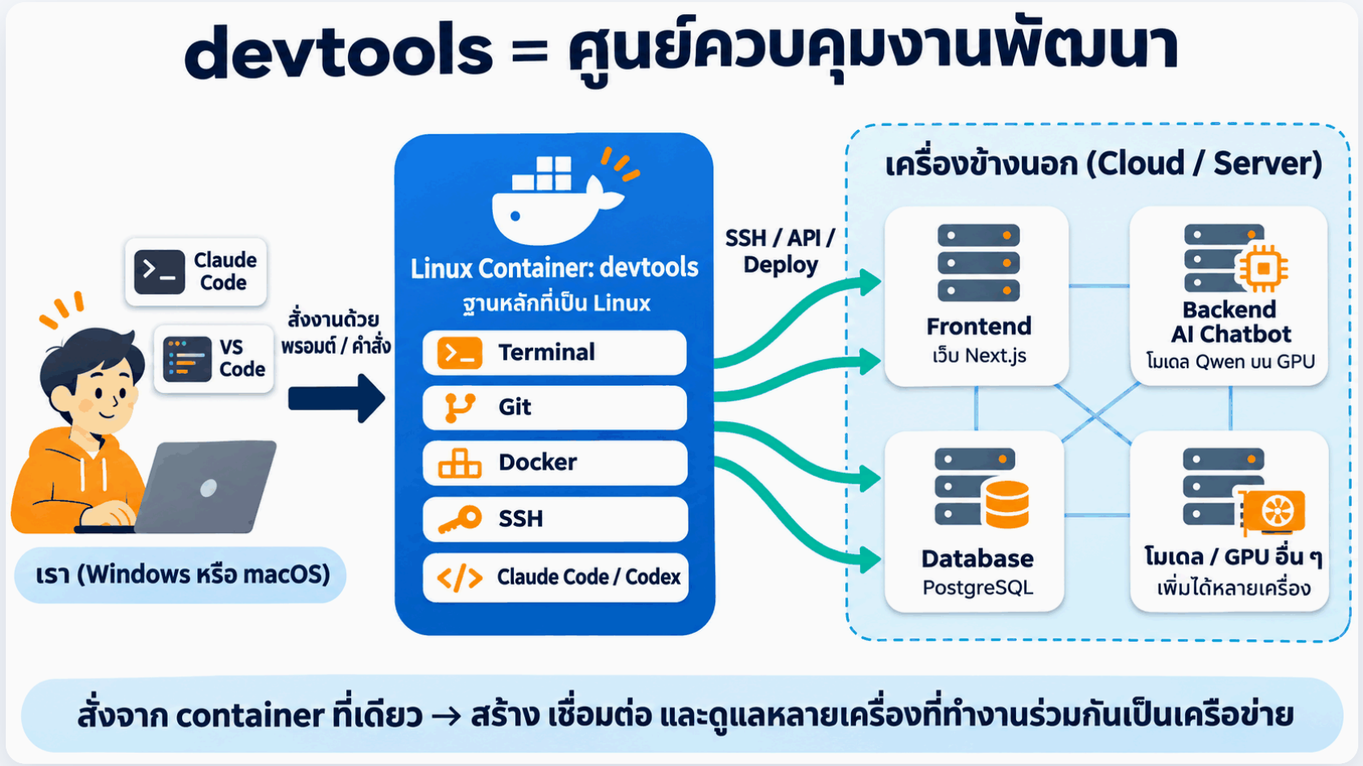


1. **Dev ในเครื่อง**
   ทำงานใน container ที่มีเครื่องมือพร้อม
2. **เรียกใช้ AI / GPU ข้างนอก**
   ส่งงานหนักไปประมวลผล แล้วรับผลกลับ
3. **Deploy ขึ้น cloud**
   server ส่วนใหญ่เป็น Linux เหมือนกัน

> container ไม่ได้ต่อ cloud ให้เอง ทุกบริการต้องมีบัญชี สิทธิ์ และ API key และมีค่าใช้จ่ายตามการใช้งาน

มองไกลกว่าห้องเรียน

devtools เป็น "ศูนย์ควบคุม" ระบบหลายเครื่องได้



1. เราสั่งงานจาก **Claude Code** หรือ **VS Code** บน Windows / Mac
2. คำสั่งไปทำงานใน **devtools** ซึ่งเป็น Linux มีเครื่องมือครบ
3. devtools รีโมทออกไปผ่าน **SSH / API / deploy** เพื่อสร้างและดูแล Frontend, Backend AI chatbot (เช่น โมเดล Qwen บน GPU), Database และโมเดลอื่น ที่เชื่อมกันเป็นเครือข่าย

> ไม่ต้องติดตั้งเครื่องมือดูแลระบบทั้งหมดบนเครื่องหลัก ทุกอย่างคุมจาก container ที่เดียว

ภาพแนวคิด สร้างด้วย Imagegen เครื่องข้างนอกเป็นตัวอย่างของระบบใน LAB ถัด ๆ ไป

ลงมือกัน

3 ขั้น ก็ได้ห้องทดลองของตัวเอง

ขั้นที่ 1

ติดตั้ง **Docker Desktop**

ขั้นที่ 2

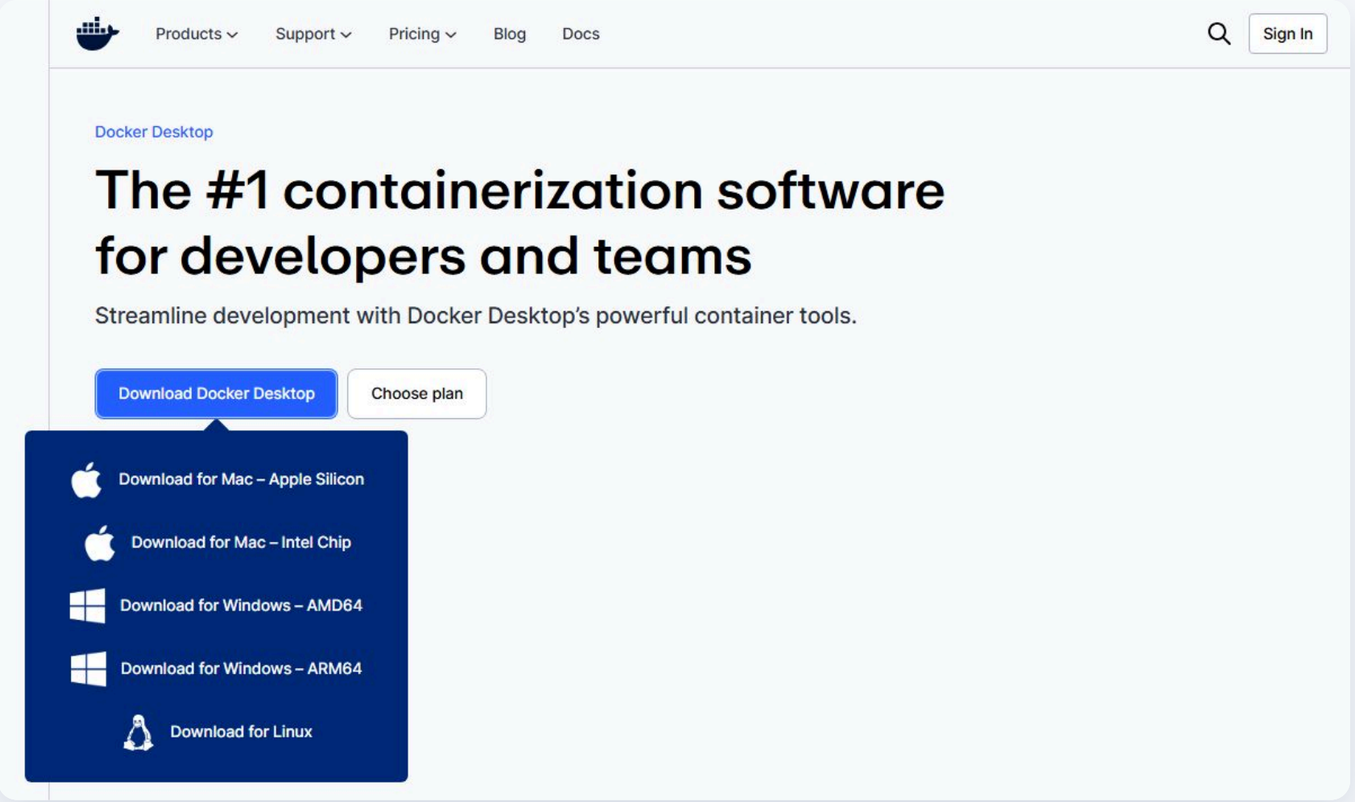
เตรียม **VS Code**

ขั้นที่ 3 · พระเอกของวันนี้

ให้ **Claude Code Desktop**
สร้าง container ให้

ขั้นที่ 1 · Docker Desktop

ดาวน์โหลด → ติดตั้งแบบ WSL 2



ภาพจริงจากเว็บไซต์ Docker

1. เปิด **docker.com/products/docker-desktop** → **Download Docker Desktop**
2. Intel / AMD เลือก **Windows – AMD64** · Windows on Arm เลือก **ARM64** (ไม่แน่ใจดู Settings → System → About → System type)
3. เปิด `Docker Desktop Installer.exe` จาก Downloads ใช้ค่าเริ่มต้น เลือก **Use WSL 2 instead of Hyper-V**
4. กด Close แล้วรีสตาร์ตถ้าระบบขอ

ขั้นที่ 1 · Docker Desktop

ดับเบิลคลิก → Engine running → ตรวจด้วย PowerShell

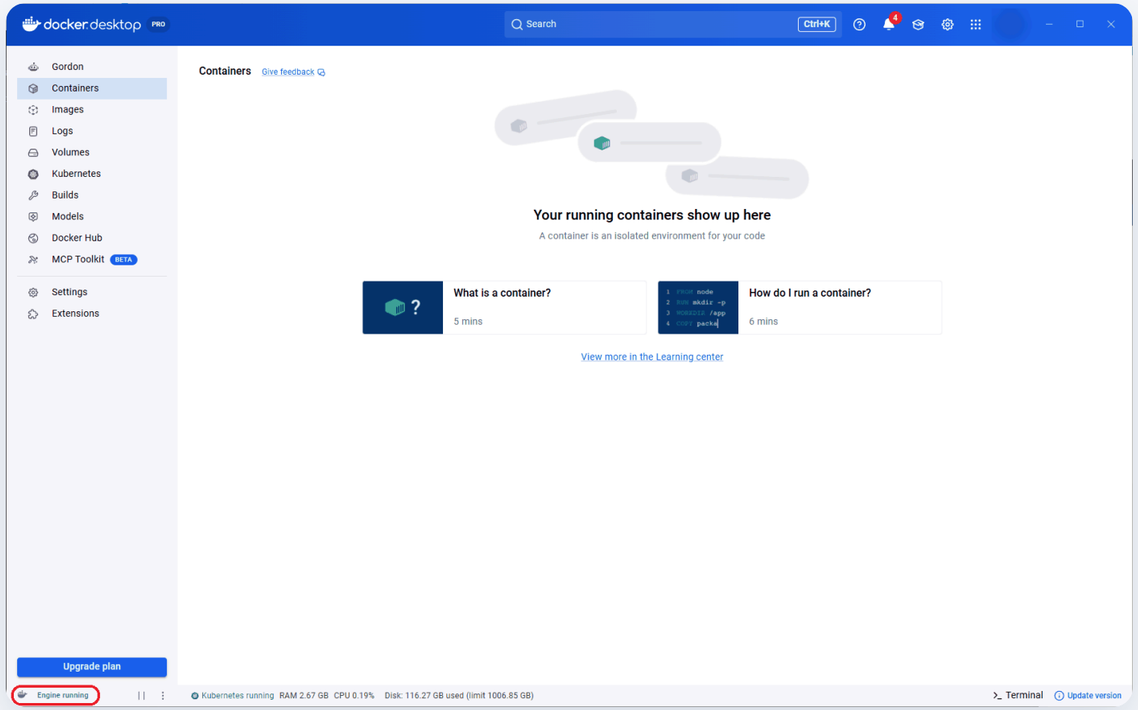


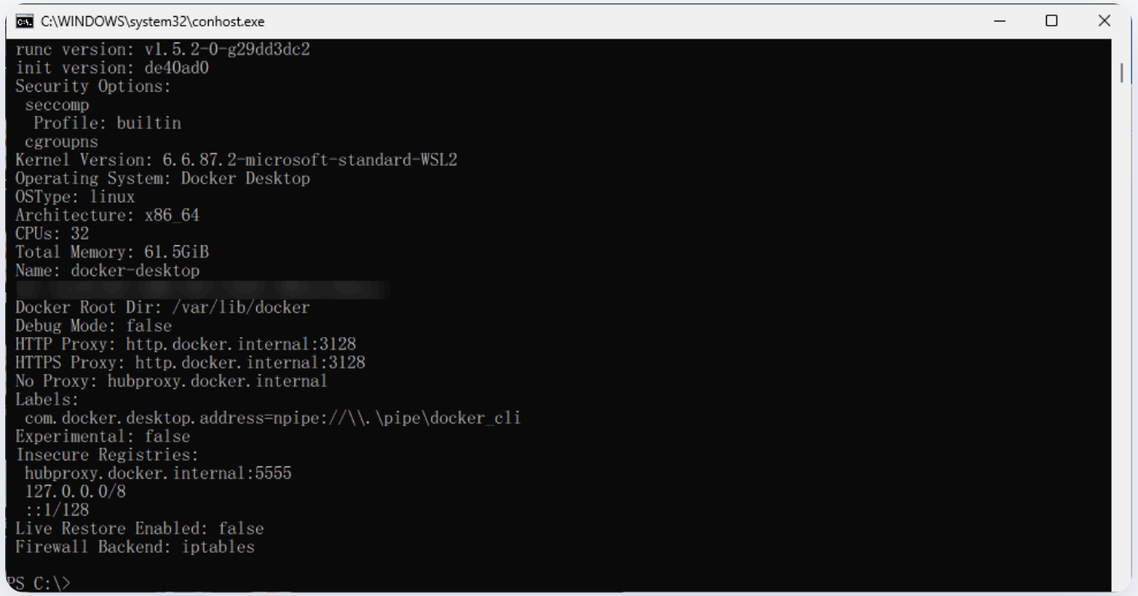
อัดจากหน้าจอจริง (ประมาณ 32 วินาที) พื้นหลังเดสก์ท็อปแทนด้วยวอลเปเปอร์ และเบลอข้อมูลส่วนตัว

ขั้นที่ 1 · ผลที่ต้องได้

Engine running + `docker info` เห็นส่วน Server

```
docker --version
docker info
```



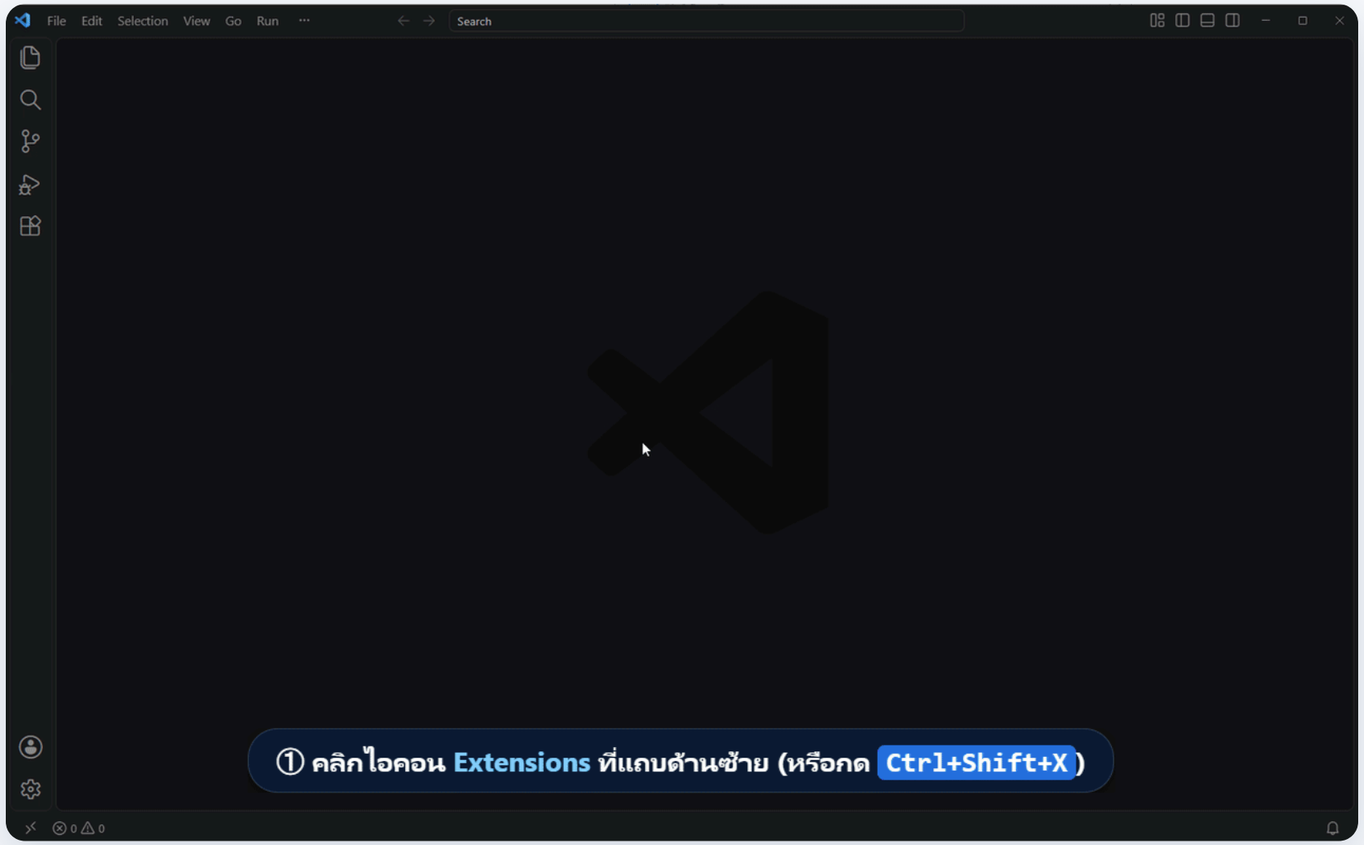


เพิ่งติดตั้ง หน้า Containers ว่างเป็นเรื่องปกติ กรอบแดง = Engine running

ถ้าขึ้น *error during connect* แปลว่า engine ยังไม่พร้อม รอจนขึ้น Engine running แล้วลองใหม่

ขั้นที่ 2 · VS Code

ติดตั้ง Dev Containers + Container Tools

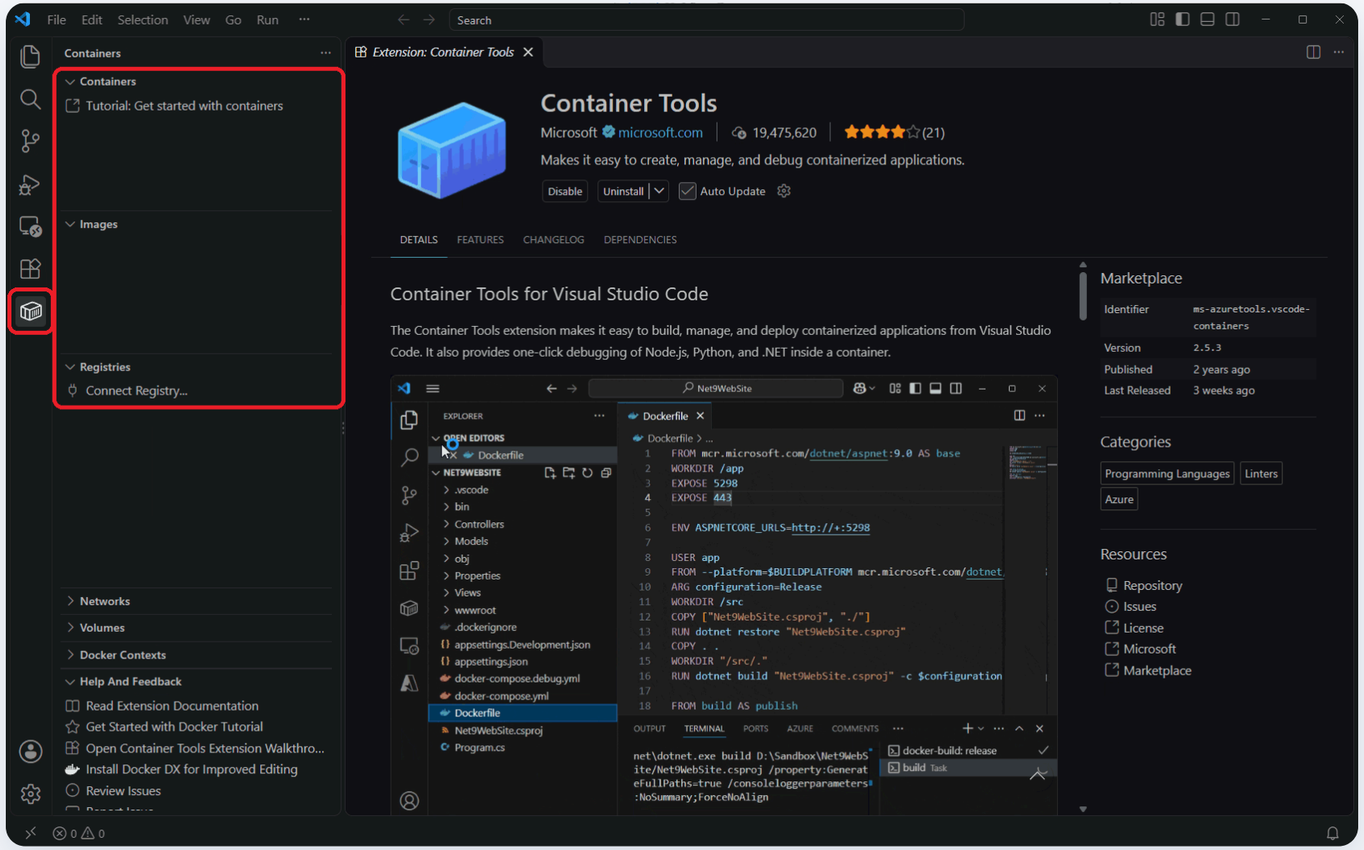


1. คลิก **Extensions** หรือกด `Ctrl+Shift+X`
2. ค้นหา **Dev Containers** ของ Microsoft → **Install**
3. ค้นหา **Container Tools** ของ Microsoft → **Install**
4. คลิก **ไอคอนรูปกล่อง** ที่แถบซ้าย

> ติดตั้ง **Claude Code** ของ Anthropic แบบเดียวกันไว้ด้วย (ใช้ในวิธีเสริม) แล้วเข้าสู่ระบบด้วยบัญชีของตนเอง

ขั้นที่ 2 · ผลที่ต้องได้

แถบ Containers ขึ้นมา = VS Code มองเห็น Docker แล้ว



เห็นหัวข้อ **Containers**, **Images**, **Registries**, **Networks**

ตอนนี้หัวข้อ Containers ยังว่าง เพราะเรายังไม่มีห้องทดลอง...

ขั้นต่อไป เราจะ พูด ให้มันเกิดขึ้น

ขั้นที่ 3 · พระเอกของวันนี้

พูดเป็นภาษาคน แล้ว container ก็เกิดขึ้นเอง

ไม่ต้องจำคำสั่ง Docker แค่วางพรอมต์ใน **Claude Code Desktop** แล้ว Claude จะสร้าง devtools ติดตั้งเครื่องมือ และทดสอบให้ดู

ขั้นที่ 3 · พรอมต์นี้ฉลาดตรงไหน

พรอมต์เดียว ใช้ได้ตั้งแต่ครั้งแรก และใช้ซ้ำได้

ระบุ image ให้ชัด: `tuchsanai/devtools:2569_1`

ยังไม่มี container

สร้าง **devtools** ใหม่จาก image แบบ privileged พร้อมเปิด port SSH และ JupyterLab

มีอยู่แล้ว

ใช้ตัวเดิม ถ้าปิดอยู่ก็เปิดให้ **ไม่ลบของเดิม**

จากนั้น

ติดตั้ง Claude Code กับ Codex ข้างใน ตรวจเวอร์ชัน และลองรันไฟล์ทดสอบ

บริการ	พอร์ตบนเครื่อง	ใน container
SSH	2222 หรือเลขว่างถัดไป	22
JupyterLab	8885 หรือเลขว่างถัดไป	8888

ก่อนเริ่ม Docker Desktop ต้องขึ้น **Engine running**

ขั้นที่ 3 · วางพรอมต์

New session → Ctrl+V → Enter




```
ช่วยเตรียม container ชื่อ devtools ในเครื่องผม จาก image
tuchsanai/devtools:2569_1

- ถ้ายังไม่มี ให้สร้างใหม่แบบ privileged (เพราะข้างในต้องใช้ Docker ได้)
  เปิด port 2222 → 22 สำหรับ SSH และ 8885 → 8888 สำหรับ JupyterLab ถ้า port
ไหนชน ให้ใช้เลขว่างถัดไป
- ถ้ามีอยู่แล้ว ให้ใช้ตัวเดิม (ถ้าปิดอยู่ให้เปิด) ห้ามลบของเดิม

จากนั้นเข้าไปทำงานข้างใน container
1. ติดตั้ง Claude Code และ Codex ถ้ายังไม่มี และทำให้พิมพ์ claude ได้เลย
2. บอกว่าข้างในเป็น Linux อะไร มี Python, Node.js, Git, Claude Code และ
Codex เวอร์ชันไหน
3. สร้างไฟล์ /workspace/hello.py ให้พิมพ์ว่า Hello from devtools แล้วลองรันให้ดู

สรุปผลเป็นตารางสั้น ๆ พร้อมบอก port ที่ใช้จริง
```

ภาพเคลื่อนไหวจำลองขั้นตอน (ประมาณ 7 วินาที) ครั้งแรกรอหลายนาที เพราะต้องดาวน์โหลด image และติดตั้งเครื่องมือ

ขั้นที่ 3 · ผลจริง (รันครั้งแรก)

ไม่ได้พิมพ์คำสั่ง Docker สักบรรทัด... แต่ได้ห้องทดลองแล้ว

Container `devtools` พร้อมใช้งานแล้ว สร้างใหม่แบบ privileged เพราะเดิมยัง

รายการ	ผล
Container	`devtools` กำลังรัน, privileged
Image	`tuchsanai/devtools:2569_1`
SSH	**2222** → 22
JupyterLab	**8885** → 8888
Linux	Ubuntu 24.04.5 LTS (Noble Numbat)
Python	3.12.3
Node.js	v22.23.2
Git	2.55.0
Claude Code	2.1.319 (ติดตั้งใหม่)
Codex	codex-cli 0.167.0 (ติดตั้งใหม่)
Docker ข้างใน	29.8.1 (daemon ตอบสนองปกติ)
`/workspace/hello.py`	รันแล้วได้ `Hello from devtools`

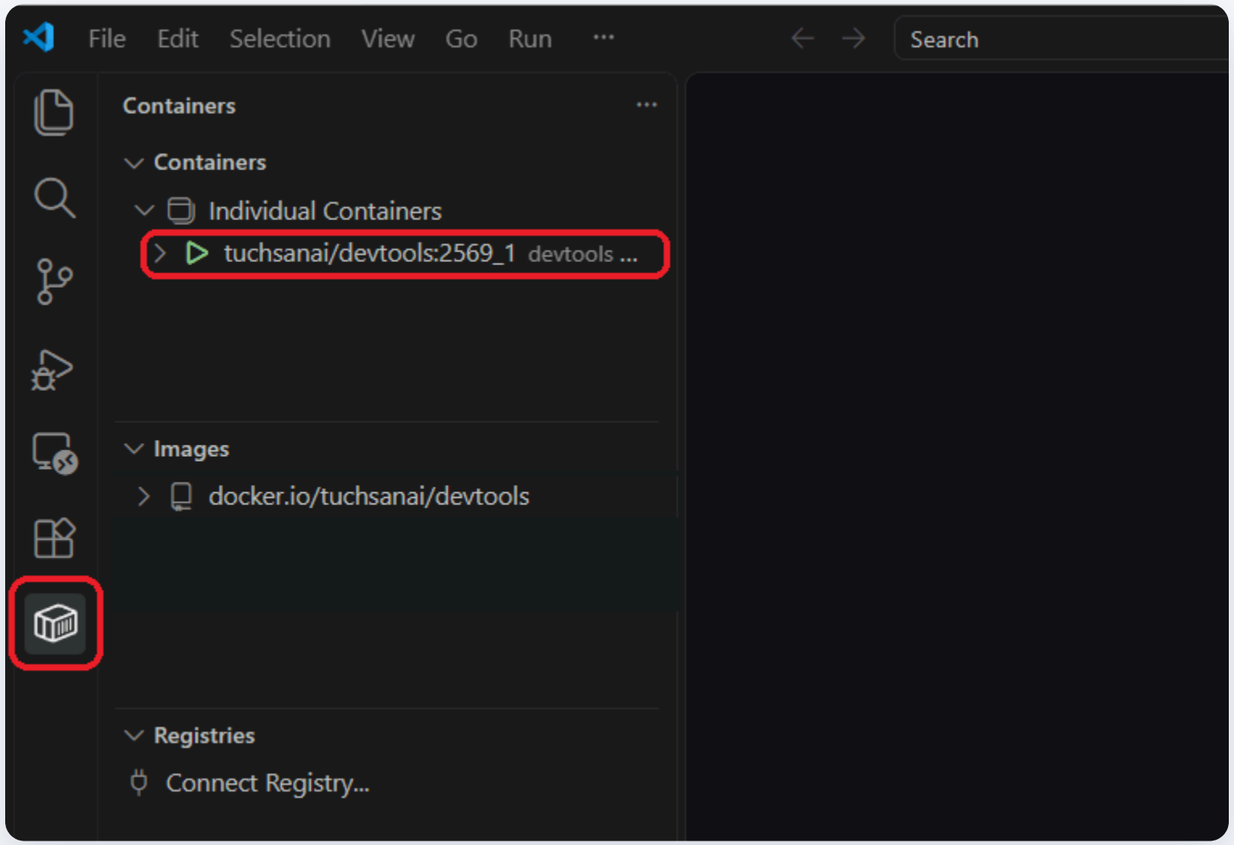
> ตอนรันยังไม่มี container ในเครื่อง → Claude สร้างใหม่จาก `tuchsanai/devtools:2569_1` แบบ privileged

- port **2222** → **22** (SSH) และ **8885** → **8888** (JupyterLab) ตามที่ขอ
- ติดตั้ง Claude Code 2.1.319 และ codex-cli 0.167.0 พิมพ์ `claude` ได้ทันที
- Ubuntu 24.04.5 · Python 3.12.3 · Node.js v22.23.2 · Git 2.55.0
- รัน `/workspace/hello.py` ได้ `Hello from devtools`

ผลจริงจาก Claude Code (Opus) บน Windows เครื่องตัวอย่าง เวอร์ชันที่ผู้เรียนได้อาจใหม่กว่า

ขั้นที่ 3 · เช็กผลใน VS Code

เปิดแถบ Containers... นี่ไง ห้องทดลองของเรา!



ใต้ **Individual Containers** เห็น `tuchsanai/devtools:2569_1 · devtools`

ไอคอน ▷ **สีเขียว** = กำลังรัน

ใต้ **Images** เห็นแม่พิมพ์ `docker.io/tuchsanai/devtools` ที่ดาวน์โหลดมา

ยังไม่เห็น กด Refresh ที่หัวแถบ และจดพอร์ตที่ใช้จริงไว้

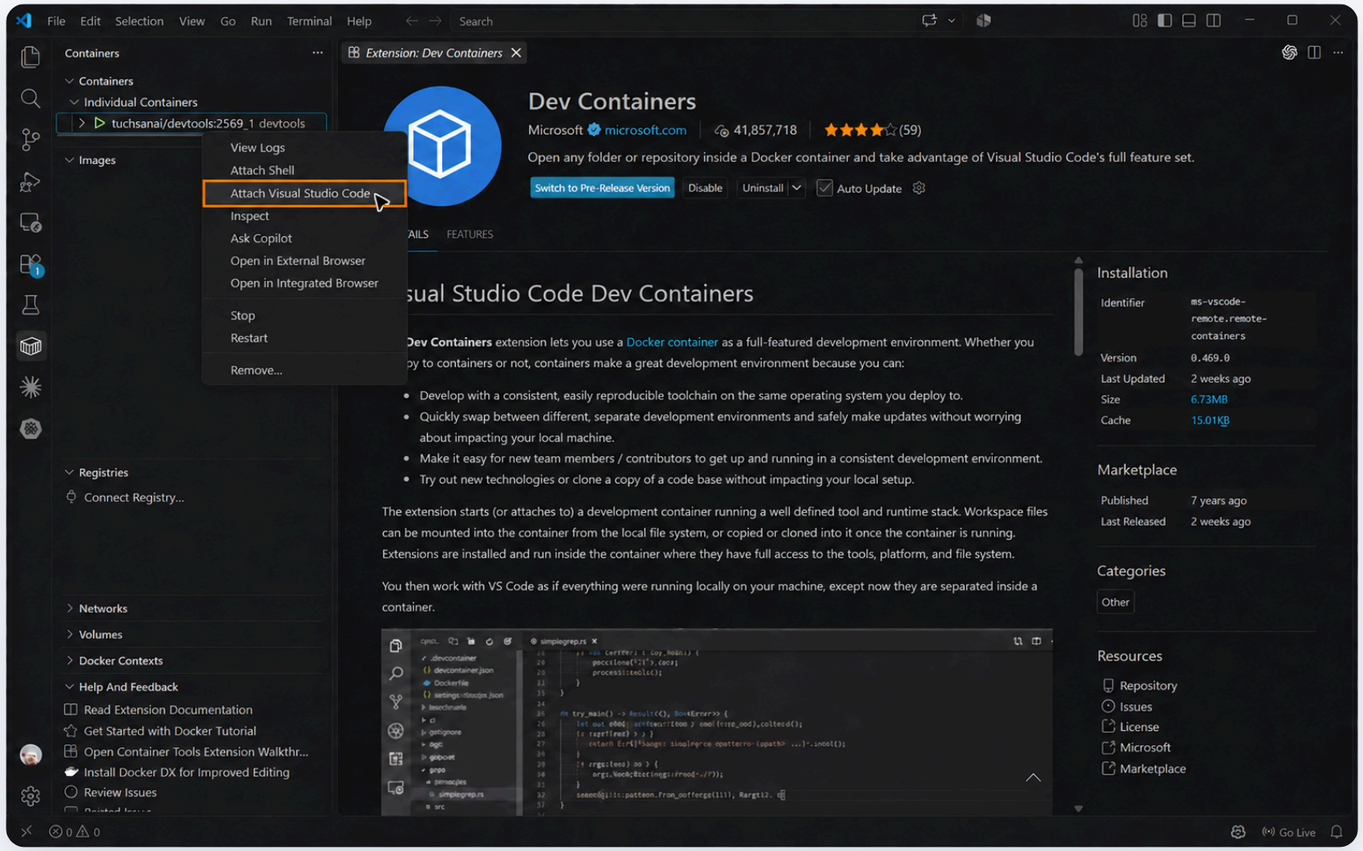
ต่อจากนี้

เข้าไปทำงานในห้องทดลอง และวิธีเสริมให้เข้าใจลึกขึ้น

เปิดประตูเข้า container ได้ 2 ทาง (VS Code และ JupyterLab) และดูเบื้องหลังว่า Claude สั่ง Docker อะไรให้เรา

ประตูที่ 1 · VS Code

คลิกขวา devtools → Attach Visual Studio Code



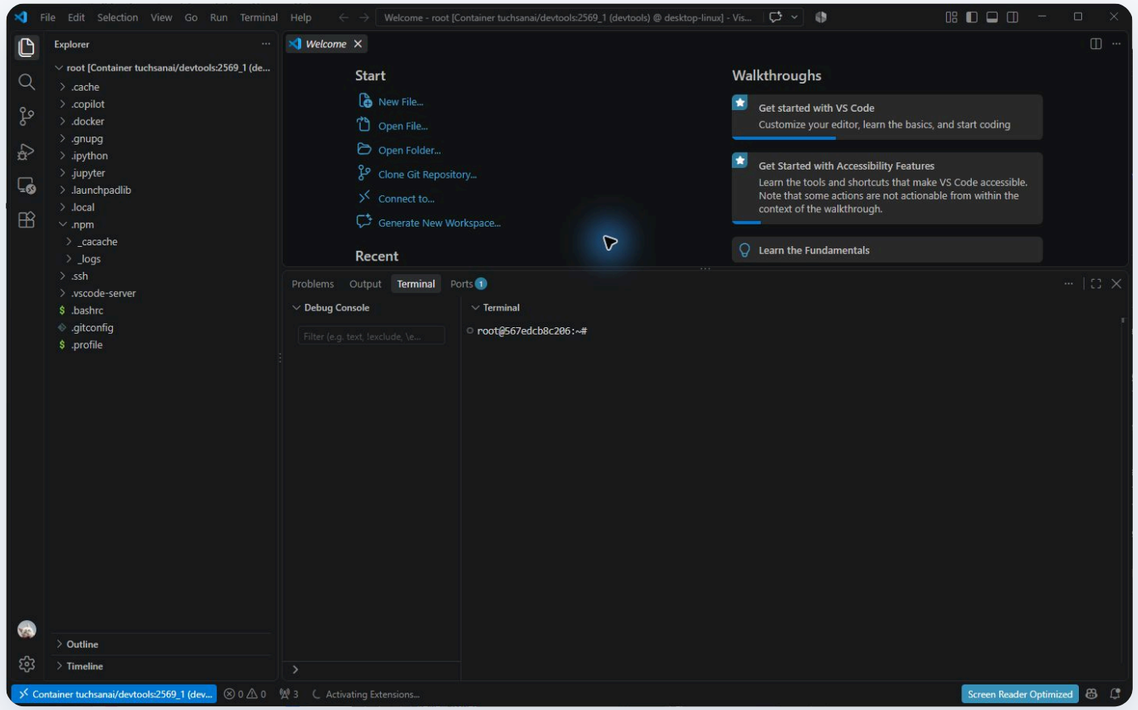
1. เปิดแถบ **Containers** หา `devtools`
2. คลิกขวา → **Attach Visual Studio Code**
3. หน้าต่างใหม่จะเชื่อมเข้า container แถบล่างซ้ายบอกว่าอยู่ใน Container

อีกวิธี: `Ctrl+Shift+P` แล้วค้นหา

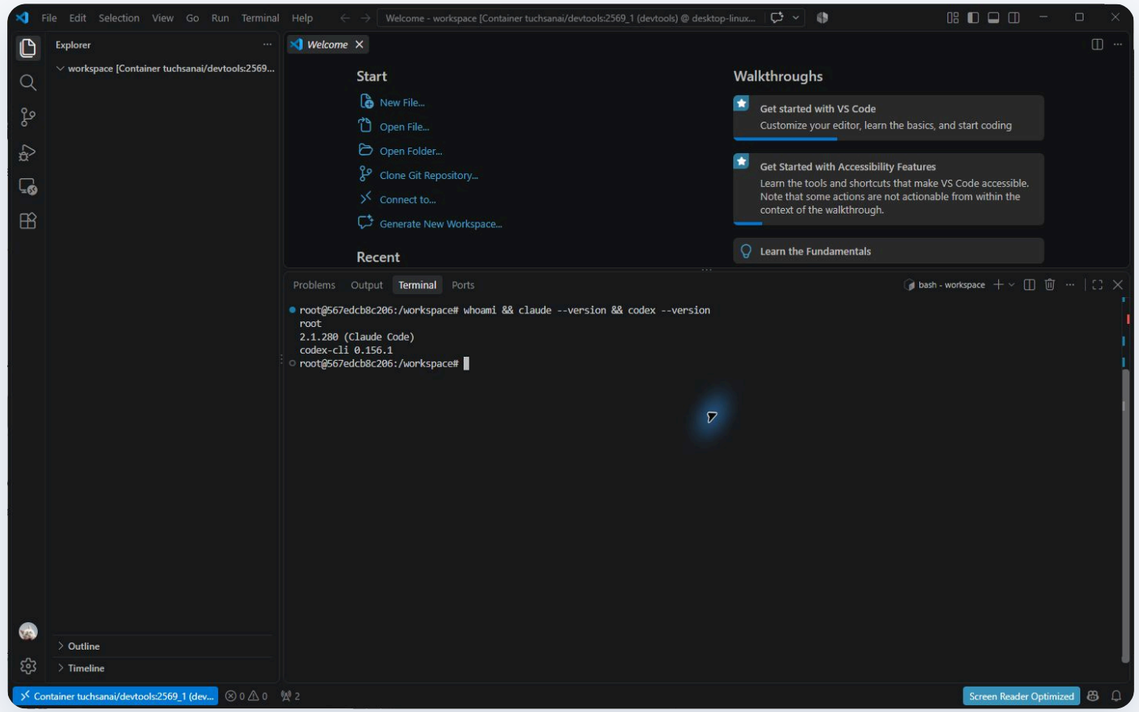
```
Dev Containers: Attach to Running Container...
```

ประตูที่ 1 · ตรวจว่าเครื่องมือพร้อม

Terminal → New Terminal แล้วรันทีละคำสั่ง




```
whoami
claude --version
codex --version
```

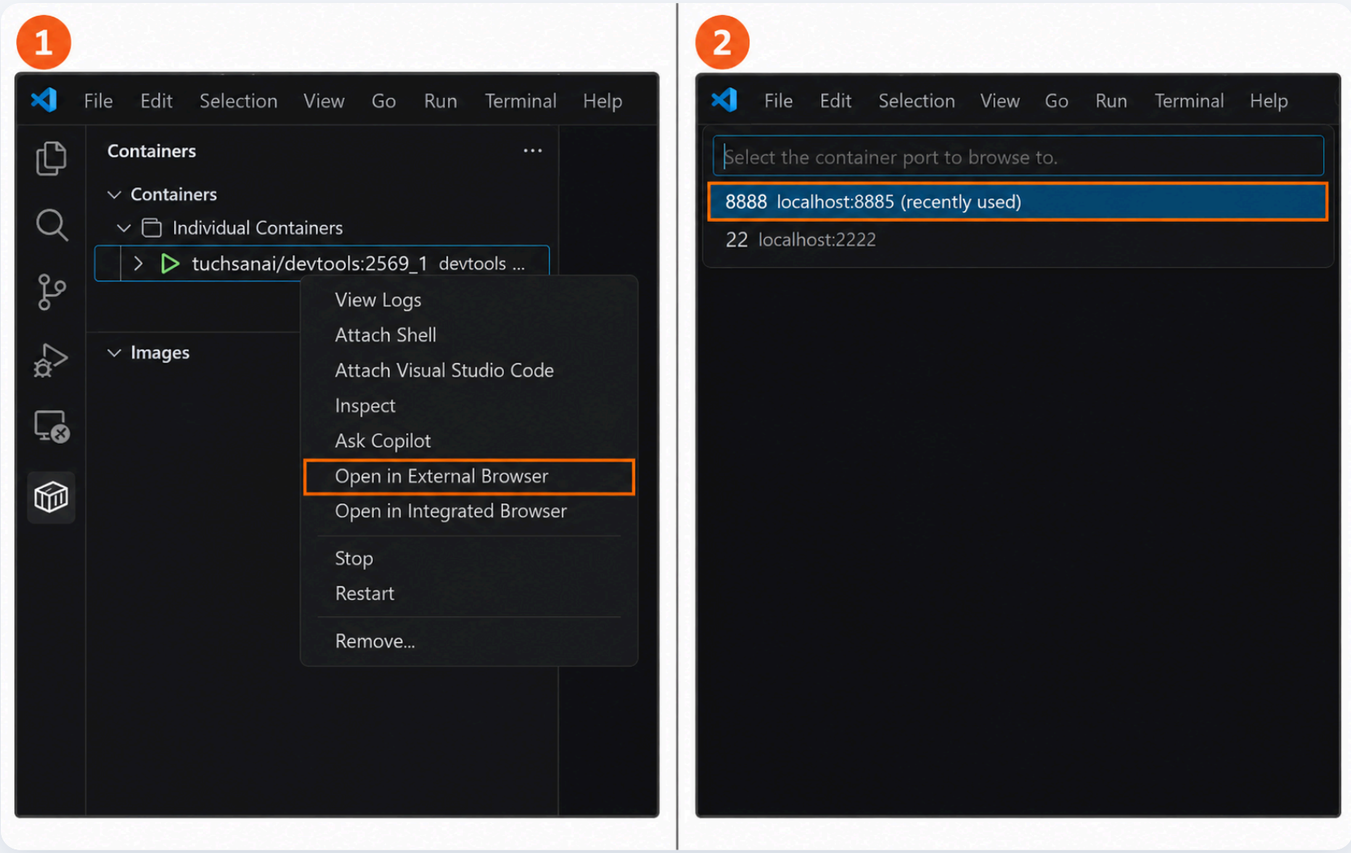


แถบล่างซ้ายต้องระบุ Container และ image ของวิชา

`whoami` ต้องได้ `root` อีกสองคำสั่งต้องแสดงเวอร์ชัน

ประตูที่ 2 · JupyterLab ในเบราว์เซอร์

คลิกขวา devtools → Open in External Browser

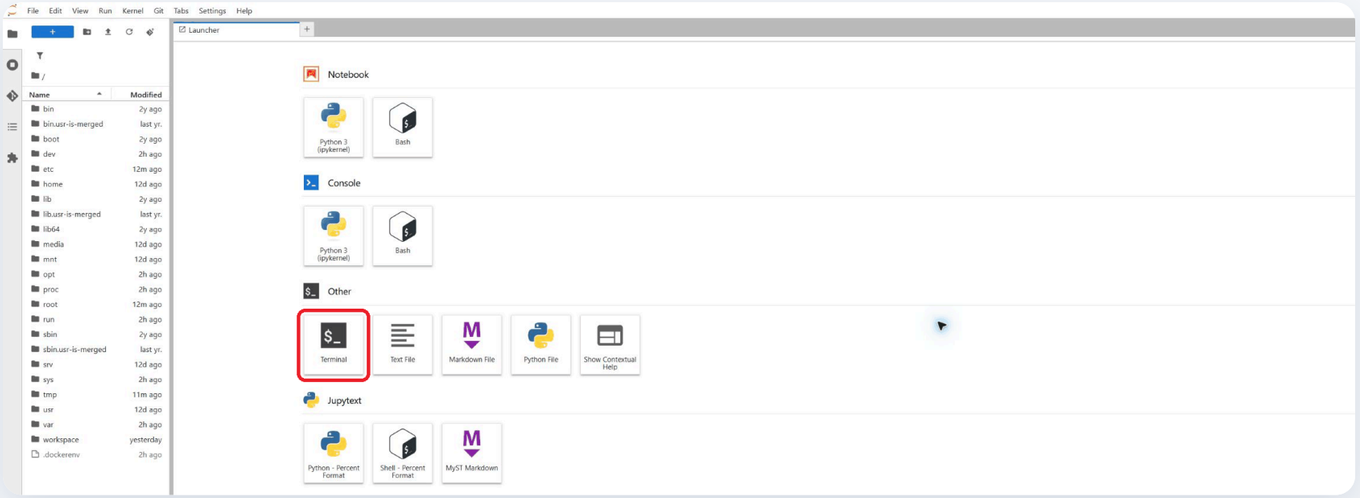


1. เลือก **8888 localhost:8885** = พอร์ต 8888 ใน container แมปมาที่ 8885 บนเครื่อง
2. เบราว์เซอร์เปิด `localhost:8885/lab`
3. ที่ Launcher เลือก **Other** → **Terminal** (ไม่เห็นกดปุ่ม **+**)

> VS Code Terminal และ JupyterLab Terminal คือ shell ของ devtools **ตัวเดียวกัน** เลือกใช้ตามสะดวก

ประตูที่ 2 · ตรวจเครื่องมือใน JupyterLab

Launcher → Terminal → ตรวจเวอร์ชัน




```
whoami
claude --version
codex --version
```

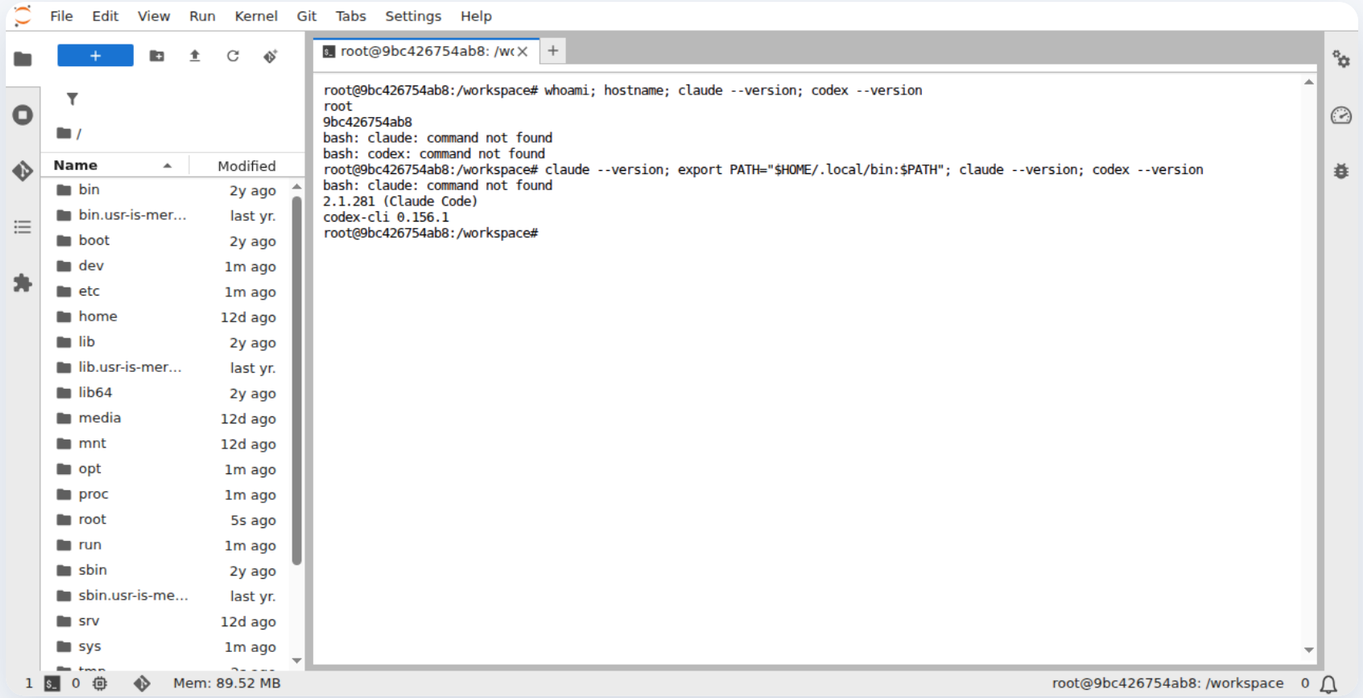
ถ้าขึ้น `claude: command not found`

```
export PATH="$HOME/.local/bin:$PATH"
claude --version
```



เข้าใจลึกขึ้น · อาการเดียวกัน สาเหตุต่างกัน

"ยังไม่ได้ติดตั้ง" กับ "ติดตั้งแล้วแต่ PATH ไม่เห็น"



1. **บรรทัดบน · ยังไม่ติดตั้ง** ไม่พบทั้ง claude และ codex → ติดตั้งให้ครบ
2. **บรรทัดกลาง · PATH ยังไม่มี** ไฟล์อยู่ที่ `~/.local/bin` แล้ว แต่ shell ยังไม่รู้จัก
3. **บรรทัดล่าง · หลัง export PATH** เห็นเวอร์ชันทั้งคู่

ตรวจเร็ว: เจอไฟล์แต่เรียกไม่ได้ = ปัญหา PATH

```
ls ~/.local/bin/claude
```

ภาพจริงจาก Terminal รันต่อเนื่องในหน้าต่างเดียว

วิธีเสริม · อยากเห็นเบื้องหลัง

สิ่งที่ Claude สั่งให้เรา หน้าตาเป็นแบบนี้

```
docker run -dit --name devtools --privileged \
  -p 2222:22 -p 8885:8888 \
  tuchsanai/devtools:2569_1

# ติดตั้ง Claude Code และ Codex ข้างใน container
docker exec -it devtools bash -c \
'curl -fsSL https://claude.ai/install.sh | bash && npm install -g
@openai/codex'
```

ตัวอย่างจากพรอมต์ในหัวข้อ 4 ของ README ใช้กับ Claude Code ใน VS Code ได้เหมือนกัน (พรอมต์นั้นลบ devtools ตัวเดิมก่อนสร้างใหม่)

`docker run`	ปั๊ม container จาก image
`--name devtools`	ตั้งชื่อห้องทดลอง
`--privileged`	ให้ใช้ Docker ข้างในได้
`-p 2222:22`	port เครื่องเรา : port ใน container (SSH)
`-p 8885:8888`	เปิด JupyterLab ที่ localhost:8885
`tuchsanai/devtools:2569_1`	แม่พิมพ์ของวิชา
`docker exec`	สั่งคำสั่งให้ไปรันข้างใน container

พร้อมใช้งาน

ในห้องทดลอง เริ่ม Claude Code หรือ Codex ได้เลย

เริ่ม Claude Code

```
claude
```

หรือเปิดอีกแท็บใช้ Codex

```
codex
```

ใช้ครั้งแรกต้อง **เข้าสู่ระบบ** ตามที่เครื่องมือแสดง การเห็นเลขเวอร์ชันยืนยันแค่ว่าติดตั้งแล้ว ยังไม่ใช่การล็อกอิน

จากนี้ไป AI จะรันคำสั่งใน **ห้องทดลองของมัน** ถ้าพลาด เราลบ container แล้วสร้างใหม่จากแม่พิมพ์ได้เสมอ

ก่อนไปทำ LAB

เช็กลิสต์ห้องทดลองของฉัน

- [ ] Docker Desktop ขึ้น **Engine running**
- [ ] `docker info` แสดงส่วน Server ได้
- [ ] VS Code มี Dev Containers + Container Tools
- [ ] วางพรอมต์ใน Claude Code Desktop สำเร็จ
- [ ] แถบ Containers เห็น **devtools** กำลังรัน
- [ ] จดพอร์ต SSH / Jupyter ที่ใช้จริงแล้ว
- [ ] Attach VS Code หรือเปิด JupyterLab เข้า devtools ได้
- [ ] `claude --version` และ `codex --version` แสดงเวอร์ชัน

คลิกช่องเพื่อติ๊กระหว่างเรียน (ไม่บันทึกถาวร)

ถ้าติดขัด

ปัญหาที่พบบ่อย

อาการ	สาเหตุและวิธีแก้
`docker info` ต่อ daemon ไม่ได้	Engine ยังไม่พร้อม รอ Docker Desktop ขึ้น **Engine running** แล้วลองใหม่
ติดตั้งแล้วแจ้งปัญหา WSL	ดูคู่มือติดตั้ง Docker Desktop สำหรับ Windows และรีสตาร์ตเครื่องหลังติดตั้ง
แถบ Containers ไม่เห็น devtools	กด **Refresh** และอ่านสรุปของ Claude ว่าสร้างสำเร็จหรือติดอะไร
พอร์ต 2222 / 8885 ถูกใช้อยู่	ใช้เลขว่างถัดไป แล้วจดเลขที่ใช้จริงไว้เปิด JupyterLab / SSH
`claude: command not found`	เช็ก `ls ~/.local/bin/claude` ถ้ามีไฟล์ให้ `export PATH="$HOME/.local/bin:$PATH"` ถ้าไม่มีให้ติดตั้งใหม่
ลบ container แล้วงานหาย	ไฟล์ใน container หายเมื่อลบ เก็บงานสำคัญด้วย Git / volume / backup ก่อนลบ



กลับไปที่คำถามแรก

40 เครื่อง คำสั่งเดียวกัน... ตอนนี้ได้ผลเหมือนกันแล้ว

ทุกคนมีห้องทดลอง Linux ชุดเดียวกัน พังได้ ลองได้ รีเซ็ตได้
และปล่อยให้ AI ลงมือทำงานในกล่องของมันได้อย่างสบายใจ 🚀

รายละเอียดทุกขั้นตอนอยู่ใน README.md · สูตรแม่พิมพ์อยู่ใน 00_Devtool_Container